

COMPILER DESIGN LABORATORY

PROJECT INSTRUCTION MANUAL

1. Introduction

The Compiler Design Laboratory Project is an integral component of the Compiler Design course. The purpose of this project is to provide students with practical exposure to the fundamental concepts involved in the design and implementation of compilers, interpreters, programming languages, and language-processing systems.

Through this project, students will apply theoretical concepts learned in the course to develop a functional software system. Students are expected to analyze a problem, design an appropriate solution, implement the required components, test the system, and demonstrate the final outcome.

The project provides practical exposure to:

- Lexical Analysis
- Syntax Analysis
- Parsing
- Semantic Analysis
- Symbol Table Management
- Intermediate Code Generation
- Code Optimization
- Target Code Generation
- Interpretation and Execution
- Error Detection and Recovery

2. Project Requirement

IMPORTANT: INDIVIDUAL PROJECT ONLY

The Compiler Design Laboratory Project is **STRICTLY AN INDIVIDUAL PROJECT**. Group projects or team submissions are not permitted.

Each student must independently:

1. Select a project topic.
2. Define the problem statement.
3. Design the proposed solution.
4. Implement the project.
5. Test and validate the system.
6. Prepare the documentation.
7. Demonstrate the project during reviews.
8. Answer technical questions related to the implementation.

Each student must be able to explain the purpose, architecture, compiler concepts, algorithms, source code, test cases, and results of the project.

3. Project Objectives

- Apply theoretical Compiler Design concepts in practical applications.
- Develop a deeper understanding of language-processing systems.
- Gain hands-on experience in designing compiler components.
- Improve programming and software development skills.
- Develop problem-solving and analytical abilities.
- Encourage independent learning.
- Promote innovation in programming languages and compiler technologies.
- Understand the complete software development lifecycle.
- Develop the ability to design, implement, test, and document a technical solution.

4. Project Topic Selection

Students are encouraged to select their own project topic. The selected project must have a clear and meaningful relationship with Compiler Design. Students are encouraged to identify innovative problems and develop practical solutions. Faculty approval is mandatory before full implementation begins.

The project proposal should include:

- Project Title
- Problem Statement
- Motivation
- Objectives
- Scope of the Project
- Compiler Design Concepts Involved
- Proposed Methodology
- System Architecture
- Tools and Technologies
- Expected Outcomes

5. Project Selection Requirements

Requirement 1: Compiler Design Relevance

The project must clearly demonstrate Compiler Design concepts.

Requirement 2: Individual Implementation

The project must be implemented independently by the student.

Requirement 3: Technical Depth

The project should demonstrate meaningful implementation. Simple input-output programs without significant Compiler Design concepts are not acceptable.

Requirement 4: Originality

Students should attempt to introduce originality through new features, improved methodology, innovative language design, unique applications, better visualization, optimization, or novel problem-solving approaches.

Requirement 5: Demonstrability

The project must produce a working demonstration.

7. Project Implementation Structure

The project will be completed in THREE IMPLEMENTATION PHASES and evaluated through THREE PROJECT REVIEWS.

Phase	Description
Phase 1	Problem Definition and System Design
Phase 2	Core Implementation
Phase 3	Final Implementation and Testing

8. PHASE 1: Problem Definition and Design

8.1 Objective

The objective of Phase 1 is to clearly define the problem and design the proposed system.

8.2 Activities

1. Topic Finalization – Select an appropriate Compiler Design-related project.
2. Problem Definition – Clearly identify the problem that the project intends to solve.
3. Background Study – Study relevant concepts, tools, algorithms, and existing systems.
4. Requirement Analysis – Identify functional, input, output, and system requirements.
5. System Design – Prepare a high-level system architecture.
6. Language Specification – Define tokens, keywords, operators, identifiers, literals, grammar rules, and syntax where applicable.
7. Technology Selection – Identify tools and technologies.
8. Prototype Development – Develop an initial working prototype.

8.3 Phase 1 Deliverables

- Project Title
- Abstract
- Problem Statement
- Motivation
- Objectives
- Scope
- Background Study
- Compiler Design Concepts Involved
- Proposed Methodology
- System Architecture

- Technology Stack
- Initial Prototype

9. REVIEW 1: PROJECT PROPOSAL AND DESIGN REVIEW

Review 1 will focus on project relevance, problem understanding, system design, methodology, technical feasibility, innovation, and initial implementation.

Criteria	Description
Topic Selection	Relevance to Compiler Design
Problem Statement	Clarity and significance
Objectives	Clearly defined objectives
Technical Feasibility	Possibility of implementation
Compiler Concepts	Appropriate concepts identified
System Architecture	Quality of design
Innovation	Originality
Prototype	Initial implementation
Viva	Individual understanding

10. PHASE 2: CORE IMPLEMENTATION

10.1 Objective

The objective of Phase 2 is to implement the major functional components of the proposed system and demonstrate substantial implementation progress.

10.2 Possible Implementation Components

- Lexical Analysis: token recognition, token classification, keyword and identifier recognition, lexical error detection.
- Syntax Analysis: grammar processing, parser implementation, syntax validation, parse tree generation.
- Abstract Syntax Tree: AST construction, traversal, and visualization.
- Semantic Analysis: type checking, scope checking, declaration validation, and semantic error detection.
- Symbol Table: symbol insertion, lookup, scope management, and data type tracking.
- Intermediate Code: three-address code, quadruples, triples, or other intermediate representations.
- Interpreter or Execution Engine: expression evaluation, program execution, runtime environment, and variable management.
- Error Handling: lexical, syntax, semantic, and runtime errors where applicable.

10.3 Phase 2 Deliverables

- Core implementation modules
- Working source code
- Functional demonstration
- Test cases
- Intermediate results
- Module-wise explanation

- Implementation progress report

11. REVIEW 2: IMPLEMENTATION PROGRESS REVIEW

Criteria	Description
Implementation Progress	Completion of planned work
Functional Correctness	Accuracy of implementation
Compiler Concepts	Proper application
Code Quality	Structure and readability
Testing	Quality of test cases
Problem Solving	Technical challenges addressed
Innovation	Improvements or unique features
Viva	Individual understanding

Students must demonstrate the working modules. Simply presenting source code without execution or demonstration is not sufficient.

12. PHASE 3: FINAL IMPLEMENTATION AND TESTING

12.1 Objective

The objective of Phase 3 is to complete the project, integrate all modules, and validate the final system.

12.2 Activities

- Complete all remaining planned functionalities.
- Integrate all modules into a complete system.
- Test using valid inputs, invalid inputs, boundary cases, error conditions, and complex inputs.
- Provide meaningful error messages wherever possible.
- Evaluate performance where applicable, including execution time, memory usage, optimization improvement, accuracy, or efficiency.
- Prepare complete project documentation.

12.3 Phase 3 Deliverables

- Complete Working Project
- Complete Source Code
- Test Cases
- Test Results
- Screenshots or Output Evidence
- Final Project Report
- Presentation
- User Instructions

13. REVIEW 3: FINAL PROJECT REVIEW

Criteria	Description
Project Completion	Completion of objectives

Functional Correctness	Correctness of outputs
Technical Complexity	Depth of implementation
Compiler Integration	Use of compiler concepts
Innovation	Originality
Code Quality	Structure and documentation
Testing	Comprehensive validation
Demonstration	Working project demonstration
Documentation	Quality of project report
Viva	Individual technical understanding

14. PROJECT ASSESSMENT RUBRIC

Component	Marks
Phase 1	10
Phase 2	10
Phase 3	10
TOTAL	100

15. Detailed Evaluation Guidelines

Review 1 – 20 Marks

Criteria	Marks
Topic Relevance	3
Problem Statement	3
Objectives	2
System Design	4
Compiler Concepts	3
Innovation	2
Prototype	3
Total	20

Review 2 – 25 Marks

Criteria	Marks
Implementation Progress	7
Technical Correctness	5
Compiler Concept Application	4
Code Quality	3
Testing	3
Individual Understanding	3
Total	25

Review 3 – 35 Marks

Criteria	Marks
Project Completion	6

Functional Correctness	6
Technical Complexity	5
Compiler Concept Integration	4
Testing	4
Demonstration	4
Viva	4
Documentation	2
Total	35

16. Innovation and Technical Complexity – 10 Marks

- Originality of the idea.
- Complexity of implementation.
- Technical contribution.
- Novel features.
- Effective application of Compiler Design concepts.

17. Documentation and Code Quality – 10 Marks

- Code structure.
- Modularity.
- Documentation.
- Comments.
- Naming conventions.
- Project organization.
- Quality of final report.

18. Academic Integrity and Originality

All students must strictly follow academic integrity guidelines.

The Following Are Prohibited

- Copying another student's source code, documentation, project architecture, or report.
- Directly downloading and submitting a project from GitHub, GitLab, or another repository as original work.
- Sharing complete project source code with other students.

Use of Open-Source Resources

Open-source libraries and frameworks may be used when appropriate. Sources must be acknowledged, the student must understand the implementation, and the project must contain meaningful individual development.

19. Use of Artificial Intelligence Tools

Students may use AI tools for learning, understanding concepts, debugging assistance, exploring algorithms, and improving documentation.

The student is fully responsible for understanding, validating, testing, modifying, and explaining all code included in the project.

- Explain the source code.
- Modify a module.
- Debug an error.
- Explain an algorithm.
- Demonstrate internal working.

20. Recommended Programming Languages

- C
- C++
- Python
- Java
- JavaScript
- TypeScript

21. Recommended Tools and Frameworks

Compiler Construction Tools

- Lex
- Flex
- Yacc
- Bison
- ANTLR

Compiler Infrastructure

- LLVM

Development Tools

- Visual Studio Code
- Eclipse
- IntelliJ IDEA
- PyCharm

22. Code Development Guidelines

- Use modular programming.
- Use meaningful variable and function names.
- Comment important logic and complex algorithms.
- Maintain a proper project folder structure.
- Create appropriate test cases.
- Maintain version history using Git where possible.
- Regularly back up the project.

- Maintain a project development log.

23. Version Control

Students are encouraged to use Git and platforms such as GitHub or GitLab to maintain version history and document development progress.

24. Project Documentation Format

9. Chapter 1: Introduction – Background, motivation, and problem area.
10. Chapter 2: Problem Statement – Clear explanation of the problem.
11. Chapter 3: Objectives – Project objectives.
12. Chapter 4: Literature Survey or Background Study – Existing approaches, technologies, and relevant compiler concepts.
13. Chapter 5: System Design – Architecture diagram, module description, data flow, and workflow.
14. Chapter 6: Methodology – Explanation of the approach used.
15. Chapter 7: Compiler Design Concepts Used – Relevant concepts such as lexical analysis, parsing, syntax trees, symbol tables, semantic analysis, intermediate code, and optimization.
16. Chapter 8: Implementation – Programming language, tools, libraries, algorithms, and major modules.
17. Chapter 9: Testing and Results – Test cases, expected and actual outputs, screenshots, and performance results.
18. Chapter 10: Conclusion – Summary of the project.
19. Chapter 11: Future Enhancements – Possible improvements.
20. References – Books, research papers, official documentation, and properly acknowledged websites.
21. Appendix – Important code details or supplementary material.

FINAL NOTE TO STUDENTS

The objective of this project is not simply to produce a large amount of code. The primary objective is to understand and apply Compiler Design concepts through meaningful implementation.

Students are encouraged to explore innovative ideas, design their own solutions, experiment with programming languages, build useful language-processing tools, and apply theoretical knowledge practically.

**THIS IS AN INDIVIDUAL PROJECT
THREE IMPLEMENTATION PHASES
THREE PROJECT REVIEWS**